

1. Міграція Legacy системи (Zero Downtime)

Тут єдиний надійний варіант — **Parallel Change** (або Expand-Contract). Робити це "в лоб" (вимкнули, мігрували, увімкнули) не можна через вимогу zero downtime.

Стратегія міграції (4 етапи):

1. **Expand (Розширення):** Я додаю нові таблиці або колонки в БД, не чіпаючи старі. Код деплоїться з логікою **Dual Write**: пишемо дані і в стару структуру, і в нову. Читаємо поки що тільки зі старої.
2. **Migrate (Міграція даних):** Пишемо фоновий скрипт, який переносить історичні дані зі старих таблиць у нові. Оскільки нові дані вже пишуться (крок 1), ми просто "наздоганяємо" історію.
3. **Contract (Перемикання):** Коли дані синхронізовані, деплоїмо зміни, щоб додаток почав **читати** з нової структури. Стара структура залишається, але стає "read-only" або ігнорується.
4. **Cleanup (Зачистка):** Коли переконалися, що все стабільно працює тиждень-два, видаляємо старий код запису і старі таблиці.

Консистентність:

Використовуємо транзакції БД. Якщо запис у нову таблицю впав — відкочуємо і запис у стару (або ретраїмо). Важливо також використовувати constraints (FK, Unique) на рівні нової схеми.

Валідація:

На етапі Dual Write я б додав у код логування розбіжностей (тіньове порівняння). Тобто, ми читаємо з обох джерел, порівнюємо, і якщо є різниця — пишемо варнінг у лог, але юзеру віддаємо старі (перевірені) дані. Плюс скрипти-звірярки (`COUNT(*)` та суми контрольних полів) після міграції.

Rollback:

Оскільки ми не видаляли старі колонки/таблиці одразу, rollback — це просто деплой попередньої версії коду, який читає зі старої схеми. Дані там актуальні, бо ми туди писали весь цей час.

2. Race Condition (Останній товар)

Проблема називається **Lost Update**. Без блокування система прочитає `stock=1` для обох, дозволить оплату, і в базі буде `-1` (або просто перезапис).

Підхід 1: Pessimistic Locking (На рівні БД)

Використовуємо `SELECT ... FOR UPDATE`.

Коли перший користувач починає транзакцію оформлення, ми блокуємо рядок з товаром.

BEGIN;

```
SELECT quantity FROM products WHERE id = 123 FOR UPDATE;  
-- Перевірка в коді: if quantity > 0 then proceed...  
UPDATE products SET quantity = quantity - 1 WHERE id = 123;  
COMMIT;
```

Другий користувач буде "висіти" і чекати, поки перший закінчить. Коли дочекається — побачить вже 0.

Плюси: Залізна надійність.

Мінуси: Знижує перформанс при високому навантаженні.

Підхід 2: Optimistic Locking (Без блокувань)

Додаємо колонку `version` у таблицю товарів. При оновленні перевіряємо, чи версія не змінилася.

```
UPDATE products  
SET quantity = quantity - 1, version = version + 1  
WHERE id = 123 AND version = 5; -- 5 - це версія, яку ми прочитали на початку
```

Якщо повернулося 0 rows updated, значить хтось встиг купити раніше. Кидаємо виключення, кажемо юзеру: "Перепрошуємо, товар закінчився".

Плюси: Швидко, не блокує базу.

3. Баг у калькуляторі знижок (Інцидент)

Технічне виявлення:

Я б написав скрипт, який бере всі замовлення за ці 3 тижні і перераховує їх вартість за *правильною* логікою (в пам'яті).

Порівнюємо `calculated_price` vs `actual_paid_price`. Складаємо список ID замовлень і дельту суми.

Refunds (Тим, хто переплатив):

Це простіше. Більшість платіжних шлюзів (Stripe, PayPal) мають API для `partial_refund`. Скрипт пробігається і повертає різницю.

Якщо картка expired: Шлюз зазвичай повертає помилку. Таким клієнтам автоматично нараховуємо цю суму (плюс бонус за незручності) на внутрішній баланс магазину або пишемо в сапорт для ручного повернення на рахунок.

Charges (Тим, хто не доплатив):

Як хороший джуніор, я б пішов до менеджера і порадив **не списувати ці гроші**.

Технічно ми можемо спробувати зробити до-списання, але:

1. Це юридично слизько (без згоди клієнта).
2. Це величезний репутаційний ризик. Втрата лояльності обійдеться дорожче ніж \$32,000.
Краще "пробачити" ці \$8/клієнт, повідомити їм про помилку, але сказати: "Це наш косяк, тому доплачувати не треба".

Комунікація:

Чесна розсилка.

1. Тим, хто переплатив: "Вибачте, ми помилилися. Гроші вже летять назад + ось вам промокод на знижку".
2. Тим, хто недоплатив: "Ми помилилися на вашу користь. Радійте, доплачувати не треба, гарного дня!"

Як запобігти:

Ввести Unit-тести саме на фінансову логіку (кейси з різними комбінаціями знижок). Такі речі не мають йти в прод без тестів. Також додати моніторинг аномалій: якщо середня знижка раптом стала 50% замість звичних 10% — алерт в Slack.

4. SaaS Архітектура (500 мерчантів)

База даних:

Для 500 мерчантів і 10k товарів (це не Big Data) робити **Database-per-tenant** — це оверкіл по інфраструктурі і коштах (500 інстансів БД менеджити важко).

Я б обрав **Shared Database** (одна БД, спільні таблиці), але з логічним розділенням.

Ізоляція даних:

В кожній таблиці (`products`, `orders`, `users`) обов'язкове поле `merchant_id`.

Для надійності я б використав **PostgreSQL Row Level Security (RLS)**. Ми створюємо політику, яка на рівні бази даних забороняє `merchant A` бачити рядки `merchant B`. Навіть якщо я, як розробник, забуду `WHERE merchant_id = ...` у SQL запиті, база не віддасть чужі дані.

Кастомізація логіки:

Без форків! Я бачу два шляхи:

1. *Простий*: Система налаштувань (JSON config), де мерчант вмикає/вимикає фічі.
2. *Складний (скрипти)*: Для кастомних знижок можна використати патерн Strategy. Дозволити мерчантам писати прості формули (наприклад, Lua-скрипти або вирази, які ми парсимо безпечно), або використовувати Webhooks. Мерчант підписується на подію `calculate_discount`, ми шлемо йому дані замовлення, він повертає суму знижки.

Shared vs Isolated:

Shared: Ядро (авторизація, білінг платформи, адмінка платформи, інфраструктура).

Isolated: Дані замовлень, клієнтів, товарів (логічно ізольовані через `merchant_id`).

Масштабування (100x):

Коли з'являється величезний клієнт, ми його "виселяємо".

Архітектура має дозволяти винести `merchant_id = 777` в окрему фізичну базу даних (**Sharding**).

У додатку буде мапінг:

- Merchants 1-499 -> DB Connection "Main"
- Merchant 500 (VIP) -> DB Connection "Dedicated"

Це дозволить не переписувати код, а просто змінити конфігурацію роутингу запитів до БД.